Chapter 4

Pioneering Specialized Hardware

IN THIS CHAPTER

» **Using standard hardware**

» **Using specialized hardware**

» **Improving your hardware**

» **Interacting with the environment**

Understanding the hardware behind AI is critical for understanding its full potential. As AI advances, the technology supporting it should evolve to meet increasing demands. This chapter explores how AI hardware has evolved, starting with general-purpose components and advancing to specialized processors tailored for complex AI applications. You determine the benefits and limitations of standard hardware, the innovations driving specialized hardware, and the computational advancements boosting AI performance.

Relying on Standard Hardware

Most AI projects at least begin with standard hardware because modern off-the-shelf components provide significant processing power, especially when compared to components from the 1980s, when AI first began to produce usable results. Consequently, even if you can't ultimately perform production-level work by using standard hardware, you can ad-

vance far enough along with your experimental and preproduction code to create a working model that will eventually process a full dataset.

Examining the standard hardware

The *architecture* (structure) of the standard PC hasn't changed since John von Neumann first proposed it in 1946. Reviewing the history at `https://lennartb.home.xs4all.nl/coreboot/col2.html` shows you that the processor connects to memory and peripheral devices through a bus in PC products as early as 1981 (and long before). All these systems use the von Neumann architecture because it provides significant benefits in modularity. Reading the history tells you that these devices allow upgrades to every component as individual decisions, allowing increases in *capability*. For example, within limits, you can increase the amount of memory or storage available to any PC. You can also use advanced peripherals. However, all these elements connect through a bus (a link between devices).



REMEMBER The PC you use today has the same architecture as devices created long ago; they're simply more capable. In addition, almost every device you can conceive of today has a similar architecture, despite having different form factors, bus types, and essential capabilities.

Describing standard hardware deficiencies

- **von Neumann bottleneck:** Of all the deficiencies, the von Neumann bottleneck is the most serious when considering the requirements of disciplines such as AI, machine learning, and even data science. You can find this particular deficiency discussed in more detail in the section "Considering the von Neumann bottleneck," later in this chapter.
- **Single points of failure:** Any loss of connectivity with the bus necessarily means that the computer fails immediately rather than gracefully. Even in systems with multiple processors, the loss of a single processor, which should simply produce a loss of capability, instead inflicts complete system failure. The same problem occurs with the loss of other system components: Rather than reduce functionality, the entire system fails. Given that AI often requires continu-

ous system operation, the potential for serious consequences escalates with the manner in which an application relies on the hardware.

- **Single-mindedness:** The von Neumann bus can either retrieve an instruction or retrieve the data required to execute the instruction, but it can't do both. Consequently, when data retrieval requires several bus cycles, the processor remains idle, further reducing its ability to perform instruction-intensive AI tasks.

EXAMINING THE HARVARD ARCHITECTURE DIFFERENCE

You may encounter the Harvard architecture during your hardware "travels" because some systems employ a modified form of this architecture to speed processing. Both the von Neumann architecture and Harvard architecture rely on a bus topology. However, when working with a von Neumann architecture system, the hardware relies on a single bus and a single memory area for both instructions and data, whereas the Harvard architecture relies on individual buses for instructions and data, and can use separate physical memory areas. The use of individual buses enables a Harvard architecture system to retrieve the next instruction while waiting for data to arrive from memory for the current instruction, thereby making the Harvard architecture both faster and more efficient. However, reliability suffers because now you have two failure points for each operation: the instruction bus and the data bus.

Microcontrollers, such as those that power your microwave, often use the Harvard architecture. In addition, you may find it in some unusual places for a specific reason. The iPhone and Xbox 360 both use modified versions of the Harvard architecture that rely on a single memory area (rather than two), but still rely on separate buses. The reason for using the architecture in this case is digital rights management (DRM). You can make the code area of memory read-only so that no one can modify it or create new applications without permission. From an AI perspective, this can be problematic because one AI's capability is to write new algorithms (executable code) as needed to deal with unanticipated situations. Because PCs rarely implement a Harvard architecture in its pure form or as the main bus construction, the Harvard architecture doesn't receive much attention in this book.

- **Tasking:** When the brain performs a task, a number of synapses fire at one time, allowing simultaneous execution of multiple operations. The original von Neumann design allowed just one operation at a time, and only after the system retrieved both the required instruction and data. Computers today typically have multiple cores, which allow simultaneous execution of operations in each core. However, application code must specifically address this requirement, so the functionality sometimes remains unused.

Relying on new computational techniques

Reading literature about how to perform tasks using AI can feel like you're hearing a marketer on TV proclaiming, "It's new! It's improved! It's downright dazzling!" So it shouldn't surprise you much that people are always coming up with ways to make the AI development experience faster, more precise, and better in other ways. The problem is that many of these new techniques are untested — they may look great, but you have to think about them for a while.

Using GPUs

After creating a prototypical setup to perform the tasks required to simulate human thought on a given topic, AI may need additional hardware to provide sufficient processing power to work with the full dataset required of a production system. Many methods are available to provide such processing power, but a common one is to use graphics processing units (GPUs) in addition to the central processor of a machine. The following sections describe the problem domain that a GPU addresses, what precisely the term GPU means, and why a GPU makes processing faster.

CONSIDERING ALAN TURING'S BOMBE MACHINE

Alan Turing's Bombe machine wasn't any form of AI. In fact, it isn't even a real computer. It broke Enigma cryptographic messages, and that's it. However, it did provide food for thought for Turing, which eventually led to a paper titled "Computing Machinery and Intelligence." Turing published that paper, which describes the imitation game, in the 1950s. (The movie *The Imitation Game* depicts the events surrounding the creation of this game.) However, the Bombe itself was actually based on a Polish machine called the Bomba.

Even though some sources imply that Alan Turing worked alone, the Bombe was produced with the help of many people, most especially Gordon Welchman. Neither did Turing spring from a vacuum, ready-made to break German encryption. His time at Princeton was spent with legendary figures like Albert Einstein and John von Neumann (who

would go on to invent the concept of computer software). The papers Turing wrote inspired these other scientists to experiment and see what is possible.

Specialized hardware of all sorts will continue to appear as long as scientists are writing papers, bouncing ideas off each other, creating new ideas of their own, and experimenting. When you see movies or other media, assuming that they're reasonably historically accurate, don't leave with the feeling that these people just woke up one morning and proclaimed, "Today, I will be brilliant!" and then went on to do something marvelous. Everything builds on something else, so history is important because it helps show the path followed and illuminates other promising paths — those not followed.

Considering the von Neumann bottleneck

The von Neumann bottleneck is a natural result of using a bus to transfer data between the processor, memory, long-term storage, and peripheral devices. No matter how fast the bus performs its task, overwhelming it — that is, forming a bottleneck that reduces speed — is always possible. Over time, processor speeds continue to increase while memory and other device improvements focus on *density* — the capability to store more in less space. Consequently, the bottleneck becomes more of an issue with every improvement, causing the processor to spend a lot of time being idle.

Within reason, you can overcome some of the issues that surround the von Neumann bottleneck and produce small, but noticeable, increases in application speed. Here are the most common solutions:

- **Caching:** When problems with obtaining data from memory fast enough with the von Neumann architecture became evident, hardware vendors quickly responded by adding localized memory that didn't require bus access. This memory appears external to the processor but as part of the processor package. High-speed cache is expensive, however, so cache sizes tend to be small.
- **Processor caching:** Unfortunately, external caches still provide insufficient speed. Even using the fastest RAM available and cutting out the bus access completely doesn't meet the processing capacity needs of the processor.

Consequently, vendors started adding internal memory — a cache smaller than the external cache, but with even faster access because it's part of the processor.

- **Prefetching:** The problem with caches is that they prove useful only when they contain the correct data. Unfortunately, cache hits prove low in applications that use a lot of data and perform a wide variety of tasks. The next step in making processors work faster is to guess which data the application will require next and load it into a cache before the application requires it.
- **Using specialty RAM:** You can get buried by RAM alphabet soup because more kinds of RAM exist than most people imagine. Each kind of RAM purports to solve at least part of the von Neumann bottleneck problem, and they do work — within limits. In most cases, the improvements revolve around the idea of getting data from memory and onto the bus faster. Two major (and many minor) factors affect speed: *memory speed* (how fast the memory moves data) and *latency* (how long it takes to locate a particular piece of data).



WARNING As with many other areas of technology, hype can become a problem. For example, *multithreading,* the act of breaking an application or other set of instructions into discrete execution units that the processor can handle one at a time, is often touted as a means to overcome the von Neumann bottleneck, but it doesn't actually help the bottleneck. Multithreading is an answer to another problem: making the application more efficient. When an application adds latency issues to the von Neumann bottleneck, the entire system slows. Multithreading ensures that the processor doesn't waste yet more time waiting for the user or the application, but instead has something to do all the time. Application latency can occur with any processor architecture, not just the von Neumann architecture. Even so, anything that speeds the overall operation of an application is visible to the user and the system as a whole.

Defining the GPU

The original intent of a GPU was to process image data quickly and then display the resulting image onscreen. During the initial phase of PC evolution, the CPU performed all the processing, which meant that graphics could appear slowly while the CPU performed other tasks. During this time, a PC typically came equipped with a *display adapter*, which contains little or no processing power. A display adapter merely converts the com-

puter data into a visual form. In fact, using just one processor proved almost impossible after the PC moved past text-only displays or extremely simple 16-color graphics. However, GPUs didn't make many inroads into computing until people began wanting 3D output. At this point, a combination of a CPU and a display adapter simply couldn't do the job.

A first step in this direction was taken by systems such as the Hauppauge 4860, which included a CPU and a special graphics chip (the 80860, in this case) on the motherboard. The 80860 provides the benefit of performing calculations extremely fast. Unfortunately, these multiprocessor, asynchronous systems didn't quite meet the expectations that people had for them (although they were incredibly fast for systems of the time), and they proved extremely expensive. Plus, there was the whole issue of writing applications that included that second (or subsequent) chip. The two chips also shared memory (which was abundant for these systems).

A GPU moves graphics processing from the motherboard to the graphics peripheral board. The CPU can tell the GPU to perform a task, and then the GPU determines the best method for doing so independently of the CPU. A GPU has a separate memory, and the data path for its bus is immense. In addition, a GPU can access the main memory for obtaining data needed to perform a task and to post results independently of the CPU. Consequently, this setup makes modern graphics displays possible.



However, what truly sets apart a GPU is that a GPU typically contains hundreds or thousands of cores contrasted with just a few cores for a CPU. Eight cores is about the best that you get, even with the newer i9 processor, an A100 GPU can host up to 80 gigabytes (GB) of RAM and has up to 8,192 FP32 (single-precision floating-point format) CUDA (Compute Unified Device Architecture) cores per full GPU. *CUDA* is a parallel computing platform and application programming interface (API) developed by NVIDIA. Even though the CPU provides more general-purpose functionality, the GPU performs calculations incredibly fast and can move data from the GPU to the display even faster. This ability is what makes the special-purpose GPU a critical component in today's systems.

Considering why GPUs work well

As with the 80860 chip, described in the previous section, GPUs now excel at performing the specialized tasks associated with graphics processing, including working with vectors. All those cores performing tasks in parallel can truly speed AI calculations.

In 2011, the Google Brain project (`https://research.google/`) trained an AI to recognize the difference between cats and people by watching movies on YouTube. However, to make this task work, Google used 2,000 CPUs in one of its giant data centers. Few people would have the resources required to replicate Google's work.

On the other hand, Bryan Catanzaro (from NVIDIA's research team) and Andrew Ng (from Stanford) were able to replicate Google's work by using a set of 12 NVIDIA GPUs. After people understood that GPUs could replace a host of computer systems stocked with CPUs, they could start moving forward with a variety of AI projects. In 2012, Alex Krizhevsky (from Toronto University) won the ImageNet computer image recognition competition using GPUs. In fact, a number of researchers have now used GPUs with amazing success.

Working with Deep Learning Processors (DLPs)

Researchers constantly struggle to discover better ways to train, verify, and test the models used to create AI applications. One of those ways is to use new computing techniques, as described in the section "Relying on new computational techniques," earlier in this chapter. Another way is to throw more processing power at the problem, such as by using a GPU.

However, a GPU is beneficial only because it can perform matrix manipulation quickly, and on a massively parallel level. Otherwise, using a GPU can create problems as well, as discussed earlier, in the "Using GPUs" section of this chapter. So the search for something better is ongoing, and you can find a veritable alphabet soup of processor types described on sites such as Primo.ai (`https://primo.ai/index.php/PRIMO.ai`) — see the page titled "Processing Units — CPU, GPU, APU, TPU, VPU, FPGA,

QPU." This resource page will acquaint you with all the current processor types. However, you should start with the overview provided in the following sections because you can easily become mired in the quicksand of facing too many options (and then your head explodes).

Defining the DLP

A *deep learning processor* (DLP) is simply a specialized processor that provides some benefits in training, verifying, testing, and running AI applications. They try to create an environment in which AI applications run quickly even on smaller or less capable devices. Most DLPs follow a similar pattern by providing

- Separate data and code memory areas
- Separate data and code buses
- Specialized instruction sets
- Large on-chip memory
- Large buffers to encourage data reuse patterns

In 2014, Tianshi Chen (and others) proposed the first DLP, called DianNoa (Chinese for *electric brain*). Of course, a first attempt is never good enough, so there's a whole family of DianNoa chips: DaDianNao, ShiDianNao, and PuDianNao (and possibly others).



REMEMBER Since these first experiments with DLPs, the number and types of DLPs have soared, but most of these endeavors are now part of university research efforts. The exceptions are the neural processing unit (NPU) created by Huawei and Samsung for mobile devices, and the tensor processing unit (TPU) created by Google (`https://cloud.google.com/tpu/docs/intro-to-tpu`) specifically for use with TensorFlow (`www.tensorflow.org`). These two DLP types are described next.

Using the mobile neural processing unit (NPU)

A number of mobile devices — notably, those by Huawei and Samsung — have a neural processing unit (NPU) in addition to a general CPU to perform AI predictive tasks using models such as artificial neural networks (ANNs) and random forests (RFs). You can't use an NPU for general computing needs because it's so specialized. However, an NPU characteristically performs up to ten times faster than a GPU does for the same task. An NPU is specialized in these ways:

- It accelerates the running of predefined models (as contrasted to training, verification, and testing).
- It's designed for use with small devices.
- It consumes little power when contrasted to other processor types.
- It uses resources, such as memory, efficiently.

Because the precise boundaries between processor types are hard to define, you might see a number of NPU look-alikes or alternatives classified as NPUs. However, here's a list of processors that you can currently classify as true NPUs:

- Ali-NPU, by Alibaba
- Ascend, by Huawei
- Neural Engine, by Apple
- Neural processing unit (NPU), by Samsung
- NNP, Myriad, EyeQ, by Intel
- NVDLA (mostly used for internet of things [IoT] devices), by NVIDIA

Accessing the cloud-based tensor processing unit (TPU)

Google specifically designed the tensor processing unit (TPU) in 2015 to more quickly run applications built on the TensorFlow framework. It represents a true chip specialization in that you can't use it effectively without TensorFlow. However, it's different in another way in that it's an application-specific integrated circuit (ASIC) rather than a full-blown CPU-type chip. The differences are important:

- An ASIC can perform only one task, and you can't change it.
- Because of its specialization, an ASIC is typically much less expensive than a CPU.
- Most ASIC implementations are much smaller than the same implementation created with a CPU.

- Compared to a CPU implementation, an ASIC is more power efficient.
- ASICs are incredibly reliable.

Creating a Specialized Processing Environment

Deep learning and AI are both non-von Neumann processes, according to many experts, including Massimiliano Versace, CEO of Neurala, Inc. (`www.neurala.com`). Because the task the algorithm performs doesn't match the underlying hardware, all sorts of inefficiencies exist, hacks are required, and obtaining a result is much harder than it should be. Therefore, designing hardware that matches the software is quite appealing. The Defense Advanced Research Projects Agency (DARPA) undertook one such project in the form of Systems of Neuromorphic Adaptive Plastic Scalable Electronics (SyNAPSE). The idea behind this approach is to duplicate nature's approach to solving problems by combining memory and processing power rather than keeping the two separate. They actually built the system (it was immense), and you can read more about it at `www.darpa.mil/news-events/2014-08-07`.

The SyNAPSE project did move forward. IBM built a smaller system by using modern technology that was both incredibly fast and power efficient. The only problem is that no one is buying them. The same holds true for IBM's SyNAPSE offering, TrueNorth. It has been hard to find people who are willing to pay the higher price, programmers who can develop software using the new architecture, and products that genuinely benefit from the chip. Consequently, a combination of CPUs and GPUs, even with its inherent weaknesses, continues to win out.

Increasing Hardware Capabilities

The CPU still works well for business systems or in applications in which the need for general flexibility in programming outweighs pure processing power. However, GPUs are now the standard for various kinds of data science, machine learning, AI, and deep learning needs. Of course, developers are constantly looking for the next big thing in the development environment. Both CPUs and GPUs are production-level processors. In the

future, you may see one of two kinds of processors used in place of these standards:

- **Application-specific integrated circuits (ASICs):** In contrast to general processors, a vendor creates an ASIC for a specific purpose. An ASIC solution offers extremely fast performance using very little power, but it lacks flexibility. You can find an example of an ASIC in this chapter in the form of a TPU (see the earlier section "Accessing the cloud-based tensor processing unit (TPU)" for details).
- **Field programmable gate arrays (FPGAs):** As with an ASIC, a vendor generally crafts an FPGA for a specific purpose. However, contrary to using an ASIC, you can program an FPGA to change its underlying functionality. An example of an FPGA solution is Microsoft's Brainwave, which is used for deep learning projects.



REMEMBER The battle between ASICs and FPGAs promises to heat up, with AI developers emerging as the winner. For the time being, Microsoft and FPGAs appear to have taken the lead. The point is that technology is fluid, and you should expect to see new developments

Vendors are also working on entirely new processing types, which may or may not work as expected. For example, Graphcore is working on an intelligence processing unit (IPU). The company has developed the line of processors shown at `www.graphcore.ai/products/ipu`. However, you have to take the news of these new processors with a grain of salt, given the hype that has surrounded the industry in the past. When you see real-life applications from large companies such as Google and Microsoft, you can start to feel a little more certain about the future of the technology involved.

Looking at the future of AI hardware, we authors see two new areas of advancement in hardware capabilities: neuromorphic computing and quantum processors. These cutting-edge technologies can potentially improve processing efficiency and capability, pushing the boundaries of AI. We look at each in turn in the following sections.

Advancing neuromorphic computing

Neuromorphic computing is the field of technology that mimics the neural structure and operation of the brain. It uses specialized hardware to create more efficient and adaptive AI systems. Recent developments in neuromorphic chips, such as Intel's Loihi (https://open-neuromorphic.org/neuromorphic-computing/hardware/loihi-intel) and IBM's TrueNorth (https://open-neuromorphic.org/blog/truenorth-deep-dive-ibm-neuromorphic-chip-design) have greatly improved power efficiency and processing capabilities for specific AI tasks.



TIP These chips can handle complex tasks — like pattern recognition, sensory processing, and real-time decision-making — with significantly lower energy consumption than traditional processors.

Neuromorphic systems often utilize *spiking neural networks* (SNNs), which process information like the brain, through the timing of spikes (bursts of activity) rather than continuous signal transmission. This approach allows for more efficient and faster data processing, especially in applications that require real-time analysis and adaptive learning.



TIP Autonomous drones and robots can benefit from neuromorphic chips by enhancing their ability to navigate and respond to their environments.

Exploring quantum processors

Quantum processors represent a big leap in AI hardware capabilities. Unlike classical processors that use bits to represent data as zeros or ones, *quantum processors* use quantum bits (qubits), which can exist in multiple states at the same time. This unique property allows quantum processors to perform certain calculations much faster than classical computers.

Companies like Google, IBM, and D-Wave are making strides in developing quantum processors. With regard to AI, quantum processors have the potential to revolutionize optimization problems, enhance machine learning algorithms, and accelerate complex simulations.

Quantum computing is still in its infancy but has several potential applications. For instance, it could enable more sophisticated AI models to process and analyze large-scale data more efficiently. This could lead to such events as drug discovery and financial modeling breakthroughs.

Adding Specialized Sensors

An essential component of AI is the capability of the AI to simulate human intelligence using a full set of senses. Input provided by senses helps humans develop the various kinds of intelligence described in Chapter 1. A human's senses provide the right sort of input to create an intelligent human. Even assuming that it becomes possible for an AI to fully implement all eight kinds of intelligence, it still requires the right sort of input to make that intelligence functional.

Humans typically have five senses with which to interact with the environment: sight, sound, touch, taste, and hearing. Oddly enough, humans still don't fully understand their own capabilities, so it's not too surprising that computers lag when it comes to sensing the environment in the same way humans do. For example, until recently, only four elements comprised taste: salt, sweet, bitter, and sour. However, two more tastes now appear on the list: umami and fat. Likewise, some women are *tetrachromats*, people who can see 100 million colors rather than the more usual 1 million. (Only women can be tetrachromats because of the chromosomal requirements.) Knowing how many women have this capability isn't even possible yet, though some sources place the number as high as 20 percent.

The use of filtered static and dynamic data now enables an AI to interact with humans in specific ways. For example, consider Alexa, the Amazon device that seemingly "hears" you and then says something in response. Even though Alexa doesn't actually understand anything you say, the ap-

pearance of communication is quite addictive and encourages people to anthropomorphize these devices. To perform any part of its task, Alexa requires access to a special sensor: a microphone that allows it to hear. Actually, Alexa has a number of microphones to help it hear well enough to provide the illusion of understanding. Unfortunately, as advanced as Alexa is, it can't see, feel, touch, or taste anything, which makes it far from human.



TIP In some cases, humans want their AI to have superior or different senses. An AI that detects motion at night and reacts to it may rely on infrared rather than normal vision. In fact, the use of alternative senses is now one of the valid uses for AI. The capability to work in environments that people can't work in is one reason that some types of robots have become popular, but working in these environments often requires that the robots have, or be connected to, a set of nonhuman sensors. Consequently, the topic of sensors actually falls into two categories (neither of which is fully defined): human-like and alternative environment.

Integrating AI with Advanced Sensor Technology

Integrating advanced sensors with AI is now changing how machines interact with the world and expanding what they can do. Here are some key advancements:

- **Sensor fusion:** Sensor fusion integrates data from multiple sensors to help understand environments. For example, combining inputs from cameras, light detection and ranging (LiDAR), radar, and ultrasonic sensors helps self-driving cars navigate safely.
- **Bio-inspired sensors:** These sensors mimic biological systems. Artificial skin detects pressure, temperature, and texture, allowing robots to handle delicate objects carefully. Other sensors can identify smells like chemical compounds, aiding in early disease detection and pollutant monitoring. (We tell you more about sensing in the next section.)
- **Quantum sensors:** Using principles of quantum mechanics, these sensors achieve high precision. They can detect tiny changes in magnetic fields, gravity, and temperature. This is important for advanced medical imaging and navigation in places where GPS doesn't work well.

- **Environmental sensors:** These sensors detect events like infrared and ultraviolet light and radiation, enabling AI systems to operate in tough conditions. They are crucial for industrial monitoring and military surveillance.

Devising Methods to Interact with the Environment

An AI that is self-contained and never interacts with the environment is useless. Of course, that interaction takes the form of inputs and outputs. The traditional method of providing inputs and outputs is directly through data streams that the computer can understand, such as datasets, text queries, and the like. However, these approaches are hardly human-friendly, and they require special skills to use.



REMEMBER Interacting with an AI is increasingly occurring in ways that humans understand better than they understand direct computer contact. For example, input occurs via a series of microphones when you ask Alexa a question. The AI turns the keywords in the question into tokens it can understand. These tokens then initiate computations that form an output. The AI tokenizes the output into a human-understandable form: a spoken sentence. You then hear the sentence as Alexa speaks to you through a speaker. In short, to provide useful functionality, Alexa must interact with the environment in two different ways that appeal to humans, but that Alexa doesn't actually understand.

Interactions can take many forms. In fact, the number and forms of interaction are increasing continually. For example, an AI can smell (see "Artificial intelligence grows a nose" at `www.science.org/content/article/artificial-intelligence-grows-nose`). However, the computer doesn't actually smell anything. Sensors provide a means to turn chemical detection into data that the AI can then use in the same way it uses all other data. The capability to detect chemicals isn't new; the ability to analyze those chemicals isn't new; nor are the algorithms used to interact with the resulting data new. What is new is the datasets used to interpret the incoming data as a smell, and those datasets come from human stud-

ies. An AI's nose has all sorts of possible uses. For example, think about the AI's capability to use a nose when working in some dangerous environments, such as to smell explosives at an airport before being able to see them by using other sensors.

Physical interactions are also on the rise. Robots that work on assembly lines are old hat, but consider the effects of robots that can drive. These are larger uses of physical interaction. Consider also that an AI can react in smaller ways. Hugh Herr, for example, uses an AI to provide interaction with an intelligent foot, as described in "Is This the Future of Robotic Legs?" at `www.smithsonianmag.com/innovation/future-robotic-legs-180953040/` and "New surgery may enable better control of prosthetic limbs" at MITNews.edu. This dynamic foot provides a superior replacement for people who have lost their real foot. Instead of the static sort of feedback that a human receives from a standard prosthetic, this dynamic foot provides the sort of active feedback that humans are used to obtaining from a real foot. For example, the amount of pushback from the foot differs when walking uphill than walking downhill. Likewise, navigating a curb requires a different amount of pushback than navigating a step.

The point is that as AI becomes more able to perform complex calculations in smaller packages with ever-larger datasets, the capability of an AI to perform interesting tasks increases. However, the tasks that the AI performs may not currently have a human equivalent. You may not ever truly interact with an AI that understands your speech, but you may come to rely on an AI that helps you maintain life — or at least make it more livable.